

Fully typed neural networks with lean

Who uses PyTorch here?

Looks familiar?

```
File "/home/danie/codice/lean-cnn/pytorch/.venv/lib/python3.12/site-pack
", line 1778, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
    ~~~~~^~~~~~
File "/home/danie/codice/lean-cnn/pytorch/.venv/lib/python3.12/site-pack
", line 1789, in _call_impl
    return forward_call(*args, **kwargs)
    ~~~~~^~~~~~
File "/home/danie/codice/lean-cnn/pytorch/mlp.py", line 15, in forward
    x = self.layer2(x)
    ~~~~~^~~~~~
File "/home/danie/codice/lean-cnn/pytorch/.venv/lib/python3.12/site-pack
", line 1778, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
    ~~~~~^~~~~~
File "/home/danie/codice/lean-cnn/pytorch/.venv/lib/python3.12/site-pack
", line 1789, in _call_impl
    return forward_call(*args, **kwargs)
    ~~~~~^~~~~~
File "/home/danie/codice/lean-cnn/pytorch/.venv/lib/python3.12/site-pack
", line 134, in forward
    return F.linear(input, self.weight, self.bias)
    ~~~~~^~~~~~
RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x30 and 29x1)
```

no more with fully typed neural networks



Formalizing Neural Networks in Lean

[Home](#)[Guide](#)[Examples](#)[Updates](#)[API Docs](#)[Graph](#)

TorchLean formalizes neural network infrastructure in Lean 4, connecting typed tensor and layer specifications, runnable training examples, graph IR semantics, floating-point contracts, CUDA trust boundaries, and artifacts that Lean checkers can inspect.

The goal is a practical bridge between modern ML workflows and formal reasoning: models can be executed, lowered, inspected, imported from PyTorch-style pipelines, and checked against explicit mathematical contracts.

<https://lean-dojo.github.io/TorchLean/>



What is Lean?

- Functional programming language and interactive theorem prover
- Every expression has a precise type — the compiler is also a proof checker
- Dependent types let you encode invariants directly in the type system
- Developed at AWS and Microsoft Research; used in formal mathematics (Mathlib)
- Growing ecosystem for verified, type-safe scientific computing

<https://lean-lang.org/>

- A Lean library providing access to PyTorch functionality
- Lives inside Lean's strong type system — with strong static guarantees
- No way to code a network with incompatible layers: shape errors caught at compile time
- Many more guarantees on the software: memory safety, totality, and formal correctness

```
-- MLP with 1 hidden layer of 30 neurons that uses ReLU activation function
def mkModel : nn.M (nn.Sequential 10 1) :=
  nn.sequential![
    nn.linear 10 30 Spec.Shape.scalar,
    nn.relu,
    nn.linear 30 1 Spec.Shape.scalar
  ]
```

Lean gets it

```
-- Input shape: one 10-D feature vector per sample.
abbrev σ : Spec.Shape := NN.Tensor.Shape.Vec 10

-- Output shape: length-1 vector (TorchLean never uses bare scalars)
abbrev τ : Spec.Shape := NN.Tensor.Shape.Vec outDim

-- MLP with 1 hidden layer of 30 neurons that uses ReLU activation
def mkModel : nn.M (nn.Sequential σ τ) :=
  nn.sequential! [
    -- nn.linear inDim 30      Spec.Shape.scalar,
    nn.linear inDim 29      Spec.Shape.scalar,
    nn.relu,
    nn.linear 30      outDim Spec.Shape.scalar
  ]
```

▼ All Messages (⊗ 1)

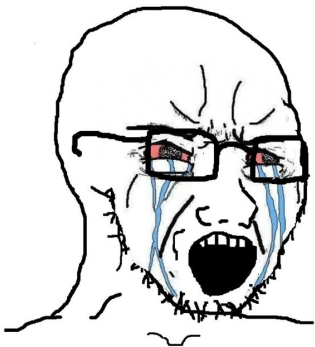
▼ Model.lean:36:2

Application type mismatch: The argument
bc†
has type
TorchLean.NN.Seq
(Spec.Shape.scalar.appendDim 30)
(Spec.Shape.scalar.appendDim outDim)
but is expected to have type
?m.60 at bc†
(Spec.Shape.scalar.appendDim 29) τ
in the application
TorchLean.NN.compAny at bc†

despite annotations this will always compile, run, and fail

```
class Mlp(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1: nn.Module = nn.Linear(10, 30)
        self.layer2: nn.Module = nn.Linear(29, 1)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.layer2(x)
        return x
```



“but annotations are annoying and useless i dont want to write them and duck typing is so convenient because flexibility or whatever”

Big duck propaganda!

- Useless annotations have been solved since the 70s, but Java wasn't paying attention
(see ML, Rust, TypeScript, Kotlin...)
- Structural type systems allow for “if it quacks like a duck...” typing
(again, see TypeScript)

Back to reality

- I'm not seriously advocating for using Lean to code neural networks
- However, stronger and flexible type systems are possible
- A man can hope for a TypeScript-like revolution for Python (and PyTorch!)

**static typing:
awesome**



**dynamic
typing: stinks**



rant over